



NATIONAL UNIVERSITY OF COMPUTER & EMERGING SCIENCES

Compiler Construction (CS4031) Mini Language Compiler

Fall 2025

Submitted To:

Miss Anaum Hamid

Group Members:

Tulaib Tausif (22K-4437)

Sharjeel Safder (22K-4210)

Abu Bakr (22K-4184)

Suhaib Sheikh (22K-4302)

Hammad Waraich (22K-4324)

Muhammad Sohaib (22K-4454)

GitHub Repository:

<https://github.com/sharjeel435/Compiler-Constructor-project.git>

1 Introduction

This report details the project carried out in our semester project of Compiler Construction with Miss Anaum Hamid as the supervisor. The aim was to code a small and custom assembly language and compile it. Under this project, we tried to practically understand the key steps of compilation and it involved lexical analysis, parsing, semantic checking, intermediate code generation, optimization and final code output.

Our language is simple and is specifically constructed that way so that we can clearly show and execute every stage of a compiler without being overly complex. This too contributed to our knowing how compilers work on actual high-level code to create a sequence of lower-level operations.

2 Mini Language Overview

The language supports basic variable declarations, arithmetic operations, and print statements. A typical example program is shown below:

```
let x = 10;
let y = x * 2 + 5;
print y;
```

The simplicity of the language makes it easy to analyze and ensures that all compiler phases remain transparent.

3 Language Specification

3.1 Lexical Rules

The lexical analyzer identifies keywords such as `let` and `print`, identifiers, numeric literals, operators (+, -, *, /, =), and punctuation symbols. Regular expressions were written for each token type, and DFA diagrams were drawn to illustrate the tokenization process.

3.2 Grammar Structure

We developed a context-free grammar that defines how statements and expressions are formed:

<program> → <stmt_list>

<stmt_list> → <stmt> <stmt_list> |

```

<stmt>          → <decl> | <assign> | <print>

<decl>          → let id = <expr> ;

<assign>        → id = <expr> ;

<print>         → print id ;

<expr>          → <expr> + <term> | <expr> - <term> | <term>

<term>          → <term> * <factor> | <term> / <factor> | <factor>

<factor>        → id | number | ( <expr> )

```

First/Follow sets and LL(1) parsing tables were constructed manually, along with parse trees for multiple test cases.

3.3 Semantic Rules

Semantic checking ensures that:

- All variables are declared before use.
- Only integer arithmetic is allowed.
- Assignments follow type-compatibility rules.

A symbol table tracks variable names, types, and assigned values.

4 Compiler Phases

4.1 Lexical Analysis

A Python tokenizer reads the input program and converts it into a sequence of tokens. Invalid characters are detected early, reducing errors in later stages. Hand-drawn DFA diagrams and token lists are included in the project submission.

4.2 Syntax Analysis

A recursive-descent parser was implemented according to our grammar. It validates the code structure and detects common syntax errors such as missing semicolons or

unbalanced parentheses. Corresponding parse trees were created for multiple sample inputs.

4.3 Semantic Analysis

During syntax traversal, a symbol table is maintained. If an undeclared variable is used, or if an invalid assignment occurs, the compiler raises a semantic error. This step ensures the logical correctness of the program.

4.4 Intermediate Code Generation

The compiler then generates Three-Address Code (TAC). For example:

```
t1 = x * 2
t2 = t1 + 5
y = t2
```

This form is easier to optimize and analyze.

4.5 Optimization

We applied several basic optimizations:

- Constant folding
- Dead code elimination
- Common subexpression elimination

These optimizations simplify the TAC and improve overall efficiency.

4.6 Final Code Generation

The optimized TAC is converted into pseudo-machine instructions. A sample output is:

```
LOAD x
MUL 2
ADD 5
STORE y
PRINT y
```

This representation shows the transformation from high-level logic to low-level operational steps.

5 System Flow

The complete compilation flow is summarized below:

1. Input program
2. Lexical analysis
3. Syntax parsing
4. Semantic checking
5. Three-Address Code generation
6. Optimization phase
7. Final code output

Each phase feeds into the next, forming a structured and modular compiler pipeline.

6 Test Cases and Results

In this section we present a few sample programs written in our mini language, along with their expected outputs. These test cases were used during development and demonstration.

Test Case 1: Simple Arithmetic

Input Program

```
let a = 10;  
let b = 5;  
let c = a + b;  
print c;
```

Expected Output

15

Test Case 2: Operator Precedence

Input Program

```
let x = 2;
let y = 3;
let z = x + y * 4;
print z;
```

Explanation

According to operator precedence, multiplication is evaluated before addition:

$$z = 2 + (3 * 4) = 2 + 12 = 14$$

Expected Output

14

Test Case 3: Reuse of Variables

Input Program

```
let n = 7;
let m = n * 2;
let k = m + n;
print m;
print k;
```

Expected Output

14

21

Test Case 4: Chained Computation

Input Program

```
let base = 4;
let height = 3;
let area = base * height / 2;
print area;
```

Explanation

This models the area of a right triangle:

$$\text{area} = \frac{4 * 3}{2} = \frac{12}{2} = 6$$

Expected Output

6

7 Example Execution (Detailed)

As a final example, we show the full flow for one program.

Input Program

```
let a = 10;  
let b = a * 2 + 5;  
print b;
```

Sample TAC

```
t1 = a * 2  
t2 = t1 + 5  
b  = t2  
print b
```

Optimized TAC

```
t1 = a * 2  
b  = t1 + 5  
print b
```

Output

25

8 Challenges and Learning

The difficulties to which we succumbed were to write a unambiguous grammar, debug the recursive parser and do a proper semantic check. These challenges notwithstanding, the project improved a lot our knowledge on the internals of compiler, symbol table, and the structures of the intermediate code.

9 Work Distribution

The project was divided into three main parts to distribute the workload fairly among the group members.

Part 1: Language Design and Lexical Analysis

By: Tulaib Tausif (22K-4437)

- Designed the mini language structure, tokens, and basic syntax.
- Implemented the lexer and created the DFA for token recognition.

Part 2: Syntax & Semantic Analysis + Intermediate Code

By: Muhammad Sohaib (22K-4454) and Hammad Waraich (22K-4324)

- Developed grammar rules, parser, and semantic checks.
- Generated Three-Address Code (TAC) for expressions and statements.

Part 3: Optimization, Code Generation, Final Integration

By: Sharjeel Safder (22K-4210) and Abu Bakr (22K-4184)

- Applied basic optimizations such as constant folding and dead code removal.
- Implemented final code generation and integrated all compiler phases.

Documentation and Handwritten Work

By: Suhaib Sheikh (22K-4302)

- Prepared DFA diagrams, parse trees, symbol tables, and handwritten documentation.

10 Conclusion

This project made us learn of the contribution of each step in a compiler to the process of the translation of the source code into the steps of executable instructions. With the guidance of Miss Anaum Hamid, we had had the chance to develop a small but practical compiler that exemplifies all the crucial stages. We would one day like to expand the language to loops, and even to the use of any mathematical expression as a boolean expression and reke whatsoever it produces as a virtual machine.

11 OUTPUT

Appendix A: Detailed Compilation Test Reports

This appendix presents the full compilation results for selected test cases. Each test includes: lexical analysis, intermediate code, optimizations, and final bytecode.

A.1 Final Compilation Summary

Description: Overall summary showing results of all 30 tests, including tokens, IR lines, optimizations, and final instructions.

```
+=====+
|                                     |
|                               FINAL COMPILATION REPORT                               |
|                                     |
+=====+

TEST RESULTS:
  Total Tests:           30
  Tests Passed:          30
  Tests Failed:           0
  Success Rate:         100.00%

OVERALL STATISTICS:
  Total Tokens Generated: 462
  Total IR Lines:         178
  Total Optimizations:    148 lines removed
  Total Instructions:     30

AVERAGES PER TEST:
  Avg Tokens:             15.40
  Avg IR Lines:           5.93
  Avg Instructions:       1.00

+=====+

[SUCCESS] All tests passed! Compiler is functioning correctly.
```

Figure 1: Final Compilation Report – All 30 tests passed successfully.

A.2 Test Case 30 – Complex Expression Evaluation

Explanation: A deeply nested arithmetic expression demonstrating correct operator precedence and heavy optimization (85% reduction).

```

OPTIMIZED CODE (After Constant Folding & DCE):
  1. z3 = 55

GENERATED INSTRUCTIONS (Bytecode):
  1. STORE 55 INTO z3

COMPILATION STATISTICS:
Tokens Generated:      19
IR Lines (Original):   7
IR Lines (Optimized):  1
Optimization Reduction: 6 lines (85.7%)
Machine Instructions:  1

=====
[+] TEST RESULT: PASS
=====

-----
TEST 30 / 30
-----

INPUT CODE:
  final = 999 - 500 - 400 + 100 - (50 / 10) * 5

EXPECTED OUTPUT:
  {'final': 174}

ACTUAL OUTPUT:
  {'final': 174}

LEXICAL ANALYSIS (Tokens):
[ 1] IDENTIFIER = final
[ 2] ASSIGN     = =
[ 3] NUMBER     = 999
[ 4] MINUS      = -
[ 5] NUMBER     = 500
[ 6] MINUS      = -

```

Figure 2: Test 30: Input, expected output, optimized IR, and final bytecode.

A.3 Test Case 28 – Division and Addition Cascade

Explanation: A chain of divisions and additions verifying left-to-right parsing and TAC correctness.

A.4 Additional Complex Expression Tests

Explanation: These tests confirm the compiler’s behavior under large chained arithmetic expressions.

A.5 Test Case 01 – Basic Assignment

Explanation: A simple variable assignment test validating correct lexical and semantic behavior.

```

[ 6] MINUS      = -
[ 7] NUMBER     = 400
[ 8] PLUS       = +
[ 9] NUMBER     = 100
[10] MINUS      = -
[11] LPAREN     = (
[12] NUMBER     = 50
[13] DIVIDE     = /
[14] NUMBER     = 10
[15] RPAREN     = )
[16] TIMES      = *
[17] NUMBER     = 5

INTERMEDIATE CODE (Three-Address Code):
1. t1 = 999 - 500
2. t2 = t1 - 400
3. t3 = t2 + 100
4. t4 = 50 / 10
5. t5 = t4 * 5
6. t6 = t3 - t5
7. final = t6

OPTIMIZED CODE (After Constant Folding & DCE):
1. final = 174

GENERATED INSTRUCTIONS (Bytecode):
1. STORE 174 INTO final

COMPILATION STATISTICS:
Tokens Generated:      17
IR Lines (Original):   7
IR Lines (Optimized):  1
Optimization Reduction: 6 lines (85.7%)
Machine Instructions:  1

=====
[+] TEST RESULT: PASS
=====

```

Figure 3: Lexical analysis and TAC breakdown for Test 30.

Overall Summary: All tests verify that the compiler performs correctly across lexical analysis, parsing, TAC generation, optimization, and instruction emission.

```

=====
[+] TEST RESULT: PASS
=====

-----
TEST 28 / 30
-----

INPUT CODE:
  z2 = 1000 / 100 / 5 + 100 / 10 / 2 + 1

EXPECTED OUTPUT:
  {'z2': 8}

ACTUAL OUTPUT:
  {'z2': 8}

LEXICAL ANALYSIS (Tokens):
[ 1] IDENTIFIER = z2
[ 2] ASSIGN     = =
[ 3] NUMBER     = 1000
[ 4] DIVIDE     = /
[ 5] NUMBER     = 100
[ 6] DIVIDE     = /
[ 7] NUMBER     = 5
[ 8] PLUS       = +
[ 9] NUMBER     = 100
[10] DIVIDE     = /
[11] NUMBER     = 10
[12] DIVIDE     = /
[13] NUMBER     = 2
[14] PLUS       = +
[15] NUMBER     = 1

```

Figure 4: Final optimized instruction and compilation statistics.

```

[11] NUMBER     = 10
[12] DIVIDE     = /
[13] NUMBER     = 2
[14] PLUS       = +
[15] NUMBER     = 1

INTERMEDIATE CODE (Three-Address Code):
  1. t1 = 1000 / 100
  2. t2 = t1 / 5
  3. t3 = 100 / 10
  4. t4 = t3 / 2
  5. t5 = t2 + t4
  6. t6 = t5 + 1
  7. z2 = t6

OPTIMIZED CODE (After Constant Folding & DCE):
  1. z2 = 8

GENERATED INSTRUCTIONS (Bytecode):
  1. STORE 8 INTO z2

COMPILATION STATISTICS:
Tokens Generated:      15
IR Lines (Original):   7
IR Lines (Optimized):  1
Optimization Reduction: 6 lines (85.7%)
Machine Instructions:  1

=====
[+] TEST RESULT: PASS
=====

```

Figure 5: Token generation, IR, and optimized output for Test 28.

```

[ 1] IDENTIFIER   = y3
[ 2] ASSIGN      = =
[ 3] NUMBER      = 10
[ 4] TIMES       = *
[ 5] NUMBER      = 10
[ 6] MINUS       = -
[ 7] NUMBER      = 20
[ 8] MINUS       = -
[ 9] NUMBER      = 30
[10] PLUS        = +
[11] NUMBER      = 50
[12] DIVIDE      = /
[13] NUMBER      = 5
[14] TIMES       = *
[15] NUMBER      = 2

INTERMEDIATE CODE (Three-Address Code):
  1. t1 = 10 * 10
  2. t2 = t1 - 20
  3. t3 = t2 - 30
  4. t4 = 50 / 5
  5. t5 = t4 * 2
  6. t6 = t3 + t5
  7. y3 = t6

OPTIMIZED CODE (After Constant Folding & DCE):
  1. y3 = 70

GENERATED INSTRUCTIONS (Bytecode):
  1. STORE 70 INTO y3

COMPILATION STATISTICS:
  Tokens Generated:      15
  IR Lines (Original):   7
  IR Lines (Optimized):  1
  Optimization Reduction: 6 lines (85.7%)
  Machine Instructions:  1

=====
+] TEST RESULT: PASS
=====

```

Figure 6: Constant folding and TAC optimization for Test 28.

```

=====+
MATHSCRIPT COMPILER - 30 COMPLEX TESTS      |
Professional Compilation Pipeline Analysis    |
=====+

-----+
TEST 01 / 30                                |
-----+

INPUT CODE:
  x = 100

EXPECTED OUTPUT:
  {'x': 100}

ACTUAL OUTPUT:
  {'x': 100}

LEXICAL ANALYSIS (Tokens):
[ 1] IDENTIFIER  = x
[ 2] ASSIGN      = =
[ 3] NUMBER      = 100

INTERMEDIATE CODE (Three-Address Code):
  1. x = 100

OPTIMIZED CODE (After Constant Folding & DCE):
  1. x = 100

GENERATED INSTRUCTIONS (Bytecode):
  1. STORE 100 INTO x

```

Figure 7: TAC and optimized output demonstrating correct precedence.

```

LEXICAL ANALYSIS (Tokens):
[ 1] IDENTIFIER  = x
[ 2] ASSIGN      = =
[ 3] NUMBER      = 100

INTERMEDIATE CODE (Three-Address Code):
1. x = 100

OPTIMIZED CODE (After Constant Folding & DCE):
1. x = 100

GENERATED INSTRUCTIONS (Bytecode):
1. STORE 100 INTO x

COMPILATION STATISTICS:
Tokens Generated:      3
IR Lines (Original):   1
IR Lines (Optimized):  1
Optimization Reduction: 0 lines (0.0%)
Machine Instructions:  1

=====
[+] TEST RESULT: PASS
=====

-----
TEST 02 / 30
-----

```

Figure 8: Test 01: Token stream, IR, optimization check, and bytecode result.